

Playwright + TypeScript Notes (Function Types & `page` Parameter)

```
import { test, Page } from "@playwright/test";
```

What is `Page`?

`Page` is one of the types provided by Playwright.

Example:

```
let page: Page;
```

Here,

- `page` → Variable name
- `Page` → Type

Just like:

```
let age: number;
```

- `age` → Variable
- `number` → Type

Different Types of Functions

```
() => {}  
  
function displayB(): void {  
  
}  
  
function displayC(a: number, b: string): void {  
  
}
```

```
function displayD(): number {  
    return 10;  
}  
  
function displayE(a: number): number {  
    return 10;  
}
```

Every function has its own **function signature**.

A function signature consists of:

- Parameters
- Parameter Types
- Return Type

Examples

```
() => void  
  
(a:number) => void  
  
(a:number,b:string) => void  
  
() => number
```

All these are different function signatures.

Generic Function Type

```
let rgb: Function;
```

`Function` is a **very generic type**.

It means:

Any function can be stored.

Examples

```
( ) => {}

(a: number) => {}

(name: string) => {}

( ) => 100

( ) => true

async( ) => {}

({page}) => {}
```

Since `Function` accepts every kind of function, it does **not** describe:

- Parameters
- Parameter Types
- Return Type

Why does `page` show an error?

```
let rgb = async ({ page }) => {  
  
}
```

When TypeScript reaches this line, it asks:

What is the type of page?

Is it

```
string?  
number?  
boolean?  
Page?
```

any?

Nobody told TypeScript.

So it reports

Binding element 'page' implicitly has an 'any' type.

Notice:

The error is about **page**, not about the function.

Why doesn't this happen inside test()?

```
test("Demo", async ({ page }) => {  
  
});
```

Here the callback is written directly inside `test()`.

Playwright already knows the callback signature.

It tells TypeScript:

```
({ page }: { page: Page }) => Promise<void>
```

Therefore

```
page = Page
```

No error.

Why does Function cause another error?

```
let rgb: Function = async ({ page }) => {
```

```
}  
  
test("Demo", rbg);
```

`test()` does **not** accept just any function.

It expects a callback having a specific signature.

Example

```
({ page }) => Promise<void>
```

But

```
Function
```

means

```
Any Function
```

Therefore

```
Function
```

is too generic for Playwright.

This is a **different error** from the `page` error.

Why does rbg2 work?

```
let rbg2 = () => {  
    console.log("Hello");  
}  
  
test("Demo", rbg2);
```

`rbg2` has

No Parameters

So TypeScript has nothing to infer.

No error.

What happens internally?

Playwright internally does something similar to this:

```
const page = await browser.newPage();

callback({

  page: page,

  context: context,

  browser: browser

});
```

Important

Playwright creates this object **at runtime**, not at compile time.

The object looks like

```
{

  page: pageObject,

  context: contextObject,

  browser: browserObject

}
```

What is Object Destructuring?

Suppose we have

```
const student = {  
  name: "Reshma",  
  age: 22  
};
```

Normally

```
const name = student.name;  
  
const age = student.age;
```

Shortcut

```
const { name, age } = student;
```

This is called

Object Destructuring

Same in Playwright

Playwright sends

```
{  
  page: pageObject,  
  context: contextObject,  
  browser: browserObject
```

```
}
```

We receive only

```
async ({ page }) => {  
    
}
```

which is equivalent to

```
async (obj) => {  
  const page = obj.page;  
}
```

So

```
{ page }
```

means

Extract only the page property
from the object.

Why do we write

```
({ page }: { page: Page })
```

There are two different `page` s.

```
({ page }: { page: Page })
```

```
|  
|  
└─ Page → Type
```

|
└─ page → Property name

Meaning

Receive an object

```
{  
  page : Page  
}  
  
and extract only  
  
page
```

Why can't we write

```
(page: {Page})
```

Because

```
{ Page }
```

is not a valid object type.

Object types must follow

```
propertyName : Type
```

Correct

```
{  
  page : Page  
}
```

Wrong

```
{  
  Page  
}
```

because TypeScript cannot understand whether

```
Page
```

is

- Property Name
or
 - Type
-

Why doesn't Playwright simply pass page?

Playwright does **not** call

```
callback(page);
```

Instead it calls

```
callback({  
  page,  
  context,  
  browser  
});
```

Since Playwright passes an **object**, we use

```
({ page })
```

to extract only the page property.

Easy Diagram

Playwright

Creates

```
{  
  page,  
  context,  
  browser  
}
```

|



```
async ({ page }) => {  
}
```

|



Extract only

page

|



page becomes a Playwright Page object.

Interview Summary

- ✓ `Page` is a Playwright type.
- ✓ `Function` is a generic function type.
- ✓ `test()` expects a callback with a specific signature.
- ✓ `page` gets an error because TypeScript doesn't know its type when the function is declared separately.
- ✓ `rbg2` works because it has no parameters.
- ✓ Playwright creates the fixtures object **at runtime**.
- ✓ `{ page }` is object destructuring.
- ✓ `{ page: Page }` means the object contains a property named `page` whose type is `Page`.
- ✓ `( { page } : { page: Page } )` means:
 - Receive an object.
 - Extract the `page` property.
 - The `page` property is of type `Page`.